

Perfeito. Vamos parar de mexer no código por enquanto e **congelar a visão do produto**.

Abaixo está um relatório simples, para você guardar e usar depois como **documento-base do API TestFlow**. Ele junta o que definimos na conversa com o que foi encontrado no framework `frameworktestes_vivopay`.

API TestFlow — Visão Funcional Consolidada

1. Objetivo do produto

O **API TestFlow** é uma plataforma web para teste automatizado de APIs.

A ideia é substituir gradualmente a operação que hoje depende de scripts, `.bat`, planilhas e arquivos JSON, mas mantendo a capacidade de executar os cenários já existentes.

O diferencial é:

o usuário configura uma API uma vez, importa uma massa de teste e manda executar. O TestFlow cuida do restante.

O TestFlow deve ser moderno, simples para o usuário e automatizado por trás.

2. Como o usuário deve pensar

O usuário **não precisa entender**:

- `pytest`;
- `token cache`;
- `JSON base`;
- `legacy_payload`;
- `contexto`;
- `estrutura interna de banco`;
- `scripts`;
- `geração de JSON`;
- `subprocessos`;
- `detalhes de execução`.

Para ele, o fluxo é:

CLIENTE

↓

TESTE DE API

↓

CONFIGURAÇÃO

↓

IMPORTAR MASSA

↓

EXECUTAR PYTEST

↓

RESULTADO

Essa é a ideia principal.

3. Cliente

O primeiro conceito do sistema para o usuário é **Cliente**.

Exemplos:

VIVO

PORTO SEGURO

ITAÚ

O cliente representa a empresa ou área para a qual os testes serão configurados.

O usuário pode:

- cadastrar cliente;
- editar cliente;
- desativar;
- excluir;
- entrar no cliente.

O cadastro do cliente deve ser simples:

Nome

Descrição

URL, autenticação, body, endpoint e massa **não pertencem ao cadastro simples do cliente**.

4. Dentro do cliente

Ao entrar em um cliente:

Clientes

↓

o usuário encontra os **Testes de API** daquele cliente.

Exemplo:

```
API Crédito
Consulta Proposta
Análise de Fraude
Homologação
```

O usuário pode:

- criar teste;
- editar teste;
- excluir;
- executar;
- consultar a última execução.

O conceito de `Projeto/Squad` pode continuar existindo internamente porque já existe na arquitetura atual, mas **não deve dominar a experiência do usuário**.

O mesmo vale para `Environment`.

5. Teste de API

Um **Teste de API** é uma configuração reutilizável para uma determinada API/processo.

Exemplo:

```
Nome:
API Crédito

Descrição:
Testes de análise de crédito - Homologação
```

A descrição pode informar o ambiente, cenário ou finalidade.

O ambiente não precisa aparecer como uma etapa obrigatória para executar.

6. Configuração do teste

Ao criar um teste de API, o usuário configura:

Informações básicas

- nome;
- descrição.

Requisição

- método HTTP;
- URL da API.

Métodos esperados:

- GET;
- POST;
- PUT;
- PATCH;
- DELETE.

Autenticação

O sistema deve oferecer:

- sem autenticação;
- Bearer Token / token informado manualmente;
- API Key;
- Basic Auth;
- autenticação automática OAuth.

O usuário deve escolher o tipo e visualizar **somente os campos necessários**.

7. Autenticação manual

Serve para o caso em que o usuário já possui um token, por exemplo obtido no Bruno/Postman.

Ele informa:

```
Bearer Token  
*****
```

O TestFlow usa esse token na chamada.

Nesse modo, o TestFlow não precisa buscar outro token.

8. Autenticação automática

É um dos recursos importantes do TestFlow.

O usuário configura uma vez os dados necessários, por exemplo:

```
URL de autenticação
Client ID
Client Secret
Grant Type
```

O TestFlow passa a:

```
verificar token
  ↓
token válido?
  ├── sim → reutiliza
  └── não → autentica novamente
```

O TestFlow atual já possui suporte a OAuth2 Client Credentials e Password, cache de token criptografado e renovação automática antes da execução. Texto colado

Também foi identificada como melhoria importante a renovação quando um token expira durante uma massa longa; essa parte deve continuar sendo tratada pelo motor, não pelo usuário. Texto colado

9. Body da request

O BODY pertence à configuração do teste/API.

Exemplo:

JSON

```
{
  "proposta": {
    "pessoa": [
      {
        "documento": "000000000000",
        "tipoPessoa": "F"
      }
    ],
    "politica": "EP_v4.1"
  }
}
```

Esse body é salvo.

O usuário não deve importar um JSON separado toda vez que for executar uma massa.

Portanto, na interface:

✗ não queremos `Importar JSON Base`.

Queremos:

Body da request

Esse body será a base para a montagem de cada caso.

10. Massa de teste

A massa representa os casos que serão executados.

O usuário deve ter um único conceito:

Importar massa

Não:

- Importar Excel legado;
- Importar JSON base;
- Importar cenário legado.

A compatibilidade com o formato antigo deve existir **internamente**.

O TestFlow atual já possui importação de Excel, preview, persistência e reutilização da massa.

Texto colado

11. Excel do framework antigo

O legado usa planilhas em que várias linhas podem representar o mesmo caso.

Exemplo:

```
caso_id | path | acao | tipo | novo_valor | ativo | observacao  
  
TC-DF-01 | proposta.pessoa[0].documento | set | string | 123456 | sim | ...  
TC-DF-01 | proposta.pessoa[0].tipoPessoa | set | string | F | sim | ...  
TC-DF-01 | proposta.politica | set | string | EP_v4.1 | sim | ...
```

O ponto importante é:

várias linhas pertencem a um único `caso_id`.

O TestFlow precisa agrupar essas linhas e gerar um payload completo para aquele caso.

Essa é uma das principais lacunas que existiam em relação ao legado.

Texto colado

12. Como a massa altera o Body

O funcionamento desejado é:

```
BODY DA CONFIGURAÇÃO
      +
MASSA
      ↓
BODY DO CASO
```

Exemplo:

Body original

JSON

```
{
  "proposta": {
    "pessoa": [
      {
        "documento": "000000000000"
      }
    ],
    "politica": "EP_v4.1"
  }
}
```

Massa

```
TC-001
proposta.pessoa[0].documento
set
12345678901
```

Payload do TC-001

JSON

```
{
  "proposta": {
    "pessoa": [
      {
```

```
"documento": "12345678901"
    }
  ],
  "politica": "EP_v4.1"
}
}
```

Cada caso começa novamente com o **BODY original**.

Um caso não pode contaminar o outro.

13. Operações da massa

A camada de compatibilidade deve entender pelo menos:

- `set` ;
- `delete` ;
- ativo/inativo;
- paths simples;
- paths profundos;
- arrays;
- números;
- strings;
- boolean;
- null.

O legado utiliza paths, arrays e alterações múltiplas por caso.

Texto colado

14. Validações

Aqui existem duas fontes de validação.

A. Validação configurada no teste

Exemplos:

```
HTTP status = 200
status = APROVADO
numero_proposta existe
```

Essas regras ficam configuradas no TestFlow.

B. Validação específica do caso

A própria massa pode representar um resultado esperado.

O framework antigo suporta `Resultado Esperado` e regras como:

- `paths;`
- `arrays;`
- `[*];`
- `null;`
- campo que deveria existir;
- campo que não deveria existir;
- divergências entre esperado e recebido. Texto colado

O TestFlow deve transformar isso em validações executáveis pelo motor atual.

15. Exemplo de validação

A API responde:

JSON

```
{
  "status": "PENDENTE"
}
```

Mas o teste espera:

```
status = APROVADO
```

Resultado:

FAIL

Observação:

Esperado status = APROVADO.

Recebido status = PENDENTE.

A observação deve ser **compreensível**, e não simplesmente:

```
AssertionError
```

O TestFlow atual já possui resultado por validação, com esperado, observado e outcome.

Texto colado

16. Fluxos encadeados

O TestFlow já suporta:

- vários steps;
- execução em sequência;
- condições;
- captura de valor da resposta;
- reutilização do valor em um step seguinte.

Texto colado

Exemplo:

```
AnaliseCredito
    ↓
pega numero_proposta
    ↓
verifica status
    ↓
se PENDENTE-FRAUDE
    ↓
AnaliseFraude
    ↓
Homologacao
```

Esse comportamento existe no legado e deve poder ser reproduzido no TestFlow.

Texto colado

17. Execução

Quando o usuário clicar:

Executar pytest

o usuário não precisa saber o que acontece internamente.

Por trás:

```
Carrega configuração
    ↓
Carrega massa
    ↓
Verifica autenticação
    ↓
Obtém/reutiliza token
    ↓
Para cada caso:
    ↓
Monta payload
    ↓
```

```
Executa requests
  ↓
Recebe response
  ↓
Executa validações
  ↓
pytest
  ↓
PASS / FAIL
  ↓
Salva evidências
  ↓
Próximo caso
```

O pytest continua sendo o mecanismo que determina o resultado formal do teste.

O TestFlow atual já executa casos individualmente usando pytest e persiste esses resultados.

Texto colado

18. Resultado

Depois da execução, o usuário deve ver automaticamente um painel.

Exemplo:

Resultado da execução

```
100 casos
92 PASS
6 FAIL
2 ATENÇÃO
```

```
Taxa de sucesso: 92%
Tempo: 2m18s
```

Depois:

ID	Descrição	Resultado	Tempo	Observação
TC-001	Cliente aprovado	PASS	320ms	Resultado conforme esperado
TC-002	CPF inválido	PASS	298ms	Resultado conforme esperado
TC-003	Proposta pendente	FAIL	410ms	Esperado APROVADO, recebido PENDENTE

O TestFlow atual já possui resumo, donut PASS/FAIL, tabela de casos, detalhes e evidências.

Texto colado

19. Detalhe / Debug

Clicando em um caso:

TC-003 — FAIL

o usuário consegue ver:

Request

- método;
- URL;
- headers;
- query params;
- body.

Response

- status HTTP;
- headers;
- body.

Validações

- regra;
- esperado;
- observado;
- resultado.

Execução

- step;
- tempo;
- erro.

O detalhe deve usar a evidência já salva.

Não deve chamar a API novamente apenas para exibir o debug.

O TestFlow já possui evidências por step e por validação.

Texto colado

20. Histórico

O sistema precisa guardar as execuções anteriores.

O usuário deve conseguir:

- procurar;
- filtrar;
- abrir uma execução;
- comparar execuções.

O histórico existente separa execuções de massa e suites e já possui filtros por cliente, projeto, ambiente e fluxo.

Texto colado

21. Regressão

O TestFlow já possui comparação entre:

```
BASELINE
x
CANDIDATA
```

A comparação é por `caso_id`.

Consegue detectar:

Regressão

```
antes = PASS
agora = FAIL
```

Correção

```
antes = FAIL
agora = PASS
```

Também identifica casos novos ou ausentes.

Texto colado

Isso é uma capacidade importante e deve ser preservada.

22. Relatórios

O TestFlow já exporta Excel.

O relatório de massa já possui:

Resumo

- total;
- PASS;
- FAIL;
- taxa;
- início;
- fim.

Casos

- caso ID;
- descrição;
- observação;
- resultado.

Evidências

- caso;
- step;
- método;
- URL;
- status;
- tempo;
- erro.

Texto colado

Posteriormente podemos complementar com:

- JSON;
- CSV;
- mais detalhes de request/response.

O legado trabalha bastante com arquivos e relatórios, então compatibilidade de saída será importante.

Texto colado

23. Dry-run

Uma funcionalidade que queremos incorporar do legado:

Validar sem chamar a API

Serve para verificar:

- Excel;

- JSON;
- estrutura da massa;
- quantidade de casos;
- paths;
- payloads;
- configuração.

Assim o usuário pode descobrir erro antes de executar uma API real.

Texto colado

24. Execução paralela

Hoje o TestFlow executa os casos sequencialmente.

O legado consegue utilizar múltiplas threads.

Para massas grandes, queremos futuramente:

- workers configuráveis;
- limite seguro;
- controle de concorrência;
- controle de rate limit;
- progresso;
- cancelamento;
- proteção do token/API.

Texto colado

Não é a primeira prioridade, mas é importante para escala.

25. Importação dos cenários existentes

O legado possui uma estrutura grande de produtos, modalidades, ambientes, regressivos e planilhas.

A análise encontrou aproximadamente:

- 15 famílias de produtos;
- ~270 arquivos `.bat`;
- ~45 planilhas de regressivo.

Texto colado

A ideia futura é permitir que esses cenários sejam migrados para o TestFlow sem recriar tudo manualmente.

O caminho sugerido é:

```
JSON Base
+
Planilha
+
Configuração
↓
Importador
↓
TestFlow
```

26. Compatibilidade com o legado

O mais importante do projeto é:

Não transformar o TestFlow em uma cópia do legado.

O legado é melhor em algumas capacidades operacionais:

- alterar requests rapidamente;
- usar planilhas de cenários;
- executar muitos casos;
- gerar arquivos;
- executar em paralelo;
- reproduzir processos específicos.

Texto colado

O TestFlow é melhor em:

- configuração;
- visualização;
- organização;
- acompanhamento;
- reutilização;
- segurança;
- histórico;
- resultados;
- arquitetura moderna.

Texto colado

Portanto:

```
LEGADO
↓
compatibilidade
↓
TESTFLOW
```

Essa é a estratégia correta.

Texto colado

27. O que NÃO devemos copiar do legado

O framework antigo possui alguns problemas que não devem ser reproduzidos:

- muitos `.bat`;
- lógica duplicada;
- código comentado;
- respostas espalhadas em pastas;
- configurações inconsistentes;
- mascaramento de dados sensíveis fraco em alguns pontos.

Texto colado

O objetivo é **migrar a funcionalidade**, não a dívida técnica.

28. Interface ideal

A experiência final deve ser simples.

Menu

```
Início
Clientes
Testes de API
Resultados
Configurações
```

Sem poluir a interface principal.

Funcionalidades como suites, regressão, histórico e relatórios continuam existindo, mas podem ficar organizadas sem dominar o fluxo principal.

29. Fluxo diário do usuário

Primeira vez

```
Clientes
↓
Novo cliente
↓
Cadastrar VIVO
```

↓
Novo teste de API
↓
Configurar URL
↓
Configurar autenticação
↓
Configurar Body
↓
Configurar validações
↓
Salvar

Uso normal

Clientes
↓
VIVO
↓
API Crédito
↓
Importar massa
↓
Validar
↓
Executar pytest
↓
Resultado

Próxima execução

VIVO
↓
API Crédito
↓
Importar nova massa
↓
Executar pytest
↓
Resultado

A configuração não precisa ser refeita.

30. O que acontece totalmente por trás

O usuário aperta **Executar pytest**.

O sistema:

1. Carrega configuração
2. Carrega body
3. Carrega massa
4. Verifica autenticação
5. Obtém/renova token quando necessário
6. Cria payload de cada caso
7. Executa requests
8. Executa steps
9. Executa condições
10. Executa validações
11. Executa pytest
12. Armazena PASS/FAIL
13. Armazena request
14. Armazena response
15. Armazena evidências
16. Gera observação
17. Consolida resultado
18. Mostra painel

O usuário vê apenas:

Executar pytest

→

Resultado

31. O que já existe x o que falta

Já existe e devemos preservar

- ✓ Cliente
- ✓ Projeto interno
- ✓ Ambiente interno
- ✓ Configuração de API
- ✓ Headers
- ✓ Autenticação
- ✓ OAuth
- ✓ Token cache
- ✓ Endpoints
- ✓ Fluxos
- ✓ Steps

- ✓ Condições
- ✓ Variáveis entre requests
- ✓ Excel
- ✓ Preview
- ✓ Massas salvas
- ✓ Execução por caso
- ✓ pytest
- ✓ PASS/FAIL
- ✓ Evidências
- ✓ Validações
- ✓ Histórico
- ✓ Regressão
- ✓ Relatórios Excel
- ✓ Suites

A análise do código confirmou praticamente todo esse núcleo.

Texto colado

Precisamos completar/melhorar

- integração natural do formato de Excel usado no legado
- montagem JSON base + massa
- Resultado Esperado vindo da massa
- configuração mais simples do Body
- dry-run
- DEBUG/painel de request-response mais completo
- JSON estruturado de resultado
- CSV compatível
- migração dos regressivos existentes
- paralelismo para massas grandes
- progresso/cancelamento
- renovação de token durante massa longa
- edição completa de algumas funcionalidades já existentes

Esses gaps foram identificados diretamente na comparação com o legado.

Texto colado

32. Prioridade de desenvolvimento

Para não virar um projeto infinito, eu congelaria esta ordem:

PRIORIDADE 1 — Uso básico

```
Cliente
→ Teste
→ Configuração
→ Body
→ Massa
→ Executar pytest
→ Resultado
```

PRIORIDADE 2 — Compatibilidade

```
Excel existente
+
JSON
→
payload por caso
```

PRIORIDADE 3 — Validação

```
Resultado Esperado
→ validação
→ PASS/FAIL
→ observação
```

PRIORIDADE 4 — Debug e relatórios

```
Request
Response
Validações
Evidências
JSON
CSV
Excel
```

PRIORIDADE 5 — Operação avançada

```
Dry-run
Paralelismo
Cancelamento
Rate limit
Migração de regressivos
CLI
```

Essa ordem também é bastante próxima da recomendação extraída do plano de evolução do legado.

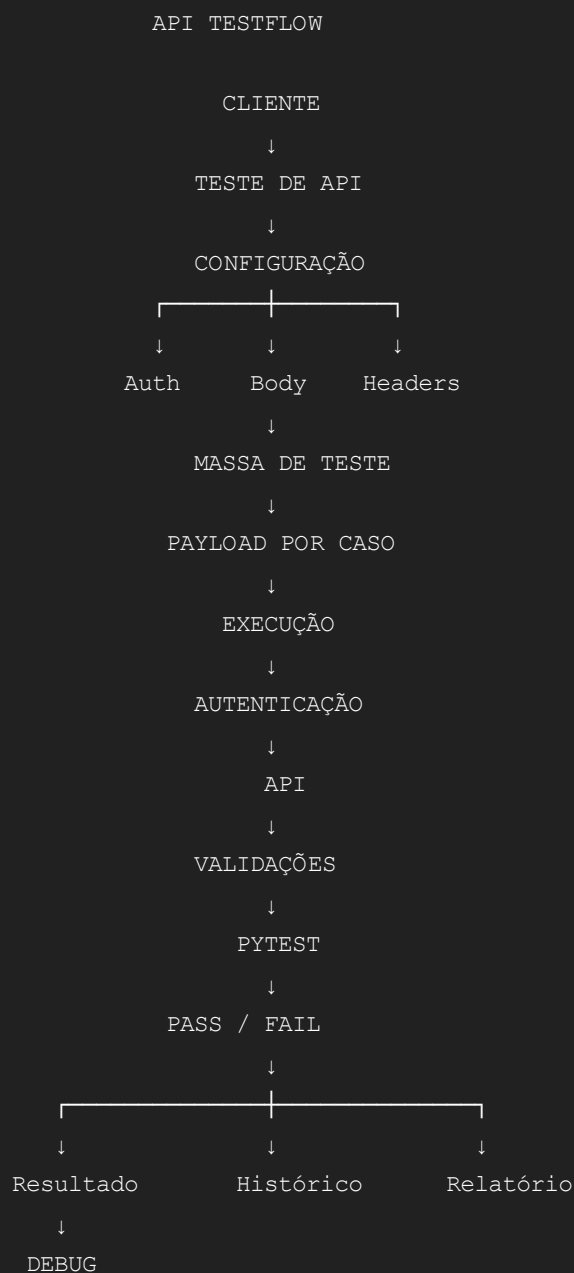
Texto colado

33. Definição simples do API TestFlow

Para guardar como referência:

API TestFlow é uma plataforma web de testes automatizados de APIs onde o usuário cadastra um cliente, configura uma API/teste uma vez, importa uma massa de cenários e executa os testes com um clique. O sistema administra autenticação, montagem das requisições, execução das APIs, validações e pytest automaticamente, apresentando os resultados, evidências, histórico, regressões e relatórios de forma simples.

E a arquitetura conceitual fica:



Esse é o documento que eu usaria como **fonte de verdade do produto** daqui para frente. A implementação atual já cobre grande parte desse núcleo; a principal necessidade é

alinhar a experiência do usuário e incorporar a compatibilidade operacional do framework existente.

Texto colado

Fontes